# Technical Document: VIB Internal Knowledge Assistant

COS40005 Computing Technology Project A · Swinburne University of Technology ·
Oct 31, 2026 · @Truong Ngoc Gia Hieu

This document specifies the technical design of the VIB Internal Knowledge Assistant, a Slack-based question-answering bot that answers employee questions from Confluence content with cited sources. It records the architecture, runtime behaviour, security model and evaluation plan, and it grounds the design choices in two published works (Boonsri & Buranasaksee 2025; Mohammed & Aref n.d.).

## 1. Introduction and Scope

### 1.1 Purpose

Staff at the sponsoring organisation (VIB) hold policy and product knowledge in Confluence, where it is slow to locate. The project, *Internal Assistant for Enterprise Knowledge (Slack App Bot)*, lets an employee ask a question in Slack and receive a short answer with links to the Confluence pages used. The system follows a retrieval-augmented generation (RAG) approach: relevant passages are retrieved first, and a large language model (LLM) answers only from those passages.

### 1.2 Scope

The work is delivered in two releases, as set out in the project Charter.

| Release | Included | Excluded or deferred |
|---|---|---|
| Alpha | Slack Bolt app in Socket Mode; channel whitelist; registered-user identity check; HTML-to-Markdown ingestion from one mock Confluence space; chunking, embedding and vector search; cited answers in English and Vietnamese; local LLM | Re-ranking, tiered access control, audit log, conversation memory, incremental sync, Word/PDF/Excel/OCR handlers, feedback buttons |
| Beta | The deferred items above | Production data (the prototype uses the mock space VIB-Mock-Knowledge only) |

Two deployment variants share one design. Project A uses free, local components (Ollama with Llama-3-8B or Phi-3). Project B targets the VIB showcase on AWS, with Amazon Bedrock as the LLM provider.

### 1.3 Document status

Items labelled *design assumption* or *proposal* were introduced by the authors of this document and have not been agreed with the client. Section 9 lists them.

### 1.4 Structure

Section 2 reviews the two works. Sections 3 to 7 specify requirements, architecture, runtime flow, security and data handling. Section 8 sets out how the system will be evaluated. Section 9 records risks and open decisions.

## 2. Background and Related Work

This section summarises the two works that inform the design and states what each contributes and what it leaves open.

### 2.1 Reference architecture for conversational agents

Mohammed and Aref (n.d.) describe a generic chatbot architecture with three core components. Natural Language Understanding (NLU) converts an utterance into a representation through topic detection, intent analysis and entity linking. The Dialogue Manager (DM) selects a response strategy, which the authors order as rule-based, knowledge-based, retrieval-based and generative. Natural Language Generation (NLG) filters candidate replies for coherence, ranks them, and returns text or speech. A context-tracking module retrieves recent utterances from a chat-history store so that references to earlier turns can be resolved. The authors identify the Dialogue Manager as the most important component because it determines how a system differs from others. Their case study reports that KLM's Messenger bot raised customer interaction by 40 per cent, but the paper contains no controlled evaluation and predates LLM-based and retrieval-augmented systems.

The paper is useful here as a shared vocabulary and as a check that the design covers each function a chatbot needs. It does not address access control, enterprise document sources or grounded generation, so it cannot be used as the design itself.

## 2.2 Repository-first, layered framework

Boonsri and Buranasaksee (2025) propose a repository-first chatbot framework paired with an AI coding agent. The framework uses a four-layer Clean Architecture (presentation, application, domain and infrastructure) with dependencies pointing inward. Its infrastructure layer uses PostgreSQL with pgvector for semantic search and Redis for session and distributed caching. New channels, data sources and LLM providers are added through small adapter interfaces, so providers can be swapped without changing the rest of the system. A reviewer applies an acceptance checklist before changes are deployed.

The authors evaluated the approach in a within-subjects experiment with nine developers, eight matched microtasks and 144 runs. Compared with an agent-only baseline, the framework condition completed tasks about 2.4 times faster (ratio 2.44, 95% CI 2.05 to 3.02) and at about 1.9 times lower cost (ratio 1.88, 95% CI 1.56 to 2.15), with fewer human interventions and comparable error rates (Boonsri & Buranasaksee 2025, Table I). A separate study with 50 users gave a mean System Usability Scale (SUS) score of 70.6, above the common benchmark of 68. The authors acknowledge limits: the study was under-powered, and for very simple tasks the agent-only condition could match or beat the framework. The headline figures also differ between the abstract (56 per cent faster) and Table I; this document uses Table I.

## 2.3 Gap addressed by this project

Neither work covers the problem addressed here: answering from an organisation's own documents while enforcing who may read what. The project therefore adopts the architectural ideas that transfer (layering, adapters, pgvector and caching, structured acceptance testing) and adds retrieval-time access control, source citation and a refusal path when no relevant passage exists. Table 1 shows how each work informs the design.

| Design element | Source of the idea | How it is used here |
|---|---|---|
| Four-layer separation with inward dependencies | Boonsri & Buranasaksee (2025) | Presentation, security, RAG core, data and storage (Section 4) |
| Adapters for LLM, store and source | Boonsri & Buranasaksee (2025) | Ollama (Project A) to Bedrock (Project B) by configuration |
| pgvector and a cache in the infrastructure layer | Boonsri & Buranasaksee (2025) | PostgreSQL with pgvector; semantic cache |
| Acceptance checklist, pass / partial / fail, SUS | Boonsri & Buranasaksee (2025) | Evaluation plan (Section 8) |

| Design element | Source of the idea | How it is used here |
|---|---|---|
| Context tracking | Mohammed & Aref (n.d.) | Conversation memory in the Beta release |
| Content filter and ranking before reply | Mohammed & Aref (n.d.) | Final access check and re-ranker (Beta) |
| Fallback when earlier strategies fail | Mohammed & Aref (n.d.) | "Not found" reply with no LLM call |
| NLU intent classification | Mohammed & Aref (n.d.) | Not used: every question follows the RAG path |

*Table 1. Mapping of the two works to the design.*

# 3. Requirements and Design Principles

## 3.1 Requirements

The requirements below are stated at the level needed to drive the design. Identifiers are local to this document (R1 to R8) and are not the Charter's numbering.

| ID | Requirement | Release |
|---|---|---|
| R1 | The bot answers only when mentioned, in a whitelisted channel | Alpha |
| R2 | Only registered users receive answers; others receive a short refusal | Alpha |
| R3 | Answers are generated only from retrieved passages and cite the Confluence pages used | Alpha |
| R4 | When no relevant passage is found, the bot says so and does not call the LLM | Alpha |
| R5 | Questions and answers may be in English or Vietnamese; the reply follows the question's language | Alpha |
| R6 | Target accuracy of at least 80% correct answers in each language, and a response time under 3 seconds | Alpha (target) |

| ID | Requirement | Release |
|---|---|---|
| R7 | A person never receives text above their permitted access level, tested at retrieval and again before reply | Beta |
| R8 | Each request writes one audit record with outcome APPROVED or DENIED | Beta |

### 3.2 Design principles

1. **Deny by default.** A request passes only if every security gate approves it. Unknown channels are ignored without a reply (design assumption).
2. **Permissions before generation.** The access filter is part of the database query, so restricted text is never retrieved and never reaches the LLM.
3. **Grounded answers.** The prompt instructs the model to use only the supplied context and to cite each source. This follows the knowledge-based and retrieval-based strategies in the reference architecture (Mohammed & Aref n.d.), with the generative step constrained by retrieved evidence.
4. **Layered separation with adapters.** Each layer depends only on the layer beneath it, and external services (LLM, vector store, embedding model, content source) are accessed through adapters, as in Boonsri and Buranasaksee (2025).
5. **Switchable Beta features.** Beta components are drawn so they can be disabled without breaking the Alpha path.
6. **Single source of truth for configuration.** Tokens are read from environment variables. Channel IDs, tier mapping and model settings live in configuration files, not in code.

## 4. System Architecture

The system is organised in four layers. A request enters at Layer 1, is checked in Layer 2, is answered in Layer 3 using data held in Layer 4, and returns the same way. Layer 4 also runs an offline ingestion path. The one-page architecture diagram and the per-layer diagrams accompany this document as separate figures.

## 4.1 Layers and components

| Layer | Components | Responsibility | Principal technology |
|---|---|---|---|
| 1. Presentation | Socket Mode client, event router, payload extractor, Block Kit renderer, Slack Web client | Receive mentions, build an event payload, acknowledge within 3 seconds, render the answer card in the thread | Slack Bolt (Python 3.11), Socket Mode (WSS), Block Kit |
| 2. Security | Ingress bridge, channel whitelist (gate 1), identity (gate 2), access tier (gate 3, Beta), egress check, audit logger (Beta) | Decide whether a request may proceed and what the person may see | whitelist.yaml, user registry table (Project A), VIB single sign-on (Project B, assumed) |
| 3. RAG core | Query router, retriever, re-ranker (Beta), generator, conversation memory (Beta) | Detect language, build the access filter, search, optionally re-rank, build the prompt, generate a cited answer | LLM adapter: Ollama (Project A) or Amazon Bedrock (Project B), temperature 0.1 |
| 4. Data ingestion and storage | Confluence connector, raw staging, format handlers, chunker, metadata injector, embedding model, vector store, semantic cache | Download, clean, split and tag pages offline; store vectors and metadata; serve filtered similarity search and cache | Confluence REST API, PostgreSQL 16 with pgvector (HNSW), JSONB metadata, Redis or in-memory cache |

*Table 2. Layers, components and technology.*

## 4.2 Correspondence with the reference architecture

The layers map onto the three core components of Mohammed and Aref (n.d.) as follows. The query router in Layer 3 plays the role of NLU, reduced to language detection and filter construction because intent classification is unnecessary when every question follows one retrieval path. The retriever and generator together play the role of the Dialogue Manager, using retrieval-based and generative strategies in sequence, with a fixed fallback reply in place of the rule-based and knowledge-base strategies. The response

stage in Layers 2 and 3 plays the role of NLG: the egress check acts as the content filter, and the re-ranker orders candidates before the reply is composed. Context tracking corresponds to conversation memory, planned for Beta.

The layering follows the four-layer structure of Boonsri and Buranasaksee (2025) in spirit but not in detail. Their layers (presentation, application, domain, infrastructure) separate software concerns of a .NET code base. The layers here separate runtime responsibilities of a retrieval pipeline, with security elevated to its own layer because access control is a central requirement of this project.

### 4.3 Interfaces between layers

- **Layer 1 to Layer 2:** an event payload containing the user ID, channel ID, thread timestamp, message text and email.
- **Layer 2 to Layer 3:** the question and a security context (user, email, tier, department, request ID). Layer 3 converts the context into a database filter.
- **Layer 3 to Layer 4:** a query vector with the filter, and cache lookups keyed by question and access scope (design assumption).
- **Layer 3 to Layer 2 and Layer 1:** an answer with citations, checked again in Layer 2 before display.

## 5. Runtime Behaviour

Seven sequence diagrams (overall and per layer) are provided in the companion document *Sequence Diagrams: Overall System and Each Layer*. This section states the behaviour they show.

### 5.1 Query flow

1. An employee mentions the bot in an approved channel. Slack delivers an `app_mention` event over the WebSocket opened by the bot, and the bot acknowledges within 3 seconds, because Slack resends events that are not acknowledged in time.
2. Layer 1 resolves the user's email and builds the event payload. Layer 2 runs gate 1 (channel), gate 2 (identity) and, in Beta, gate 3 (tier). Any failure ends the request with a refusal and an audit record.
3. Layer 3 detects the question's language, forms the access filter from the security context and checks the semantic cache. A hit returns the stored answer and skips the remaining steps.
4. On a miss, the retriever embeds the question and requests the 50 nearest chunks that satisfy the filter. In Beta, a cross-encoder re-ranker narrows these to three.

5. If no chunk exceeds the relevance threshold, the system returns a fallback reply in the question's language and does not call the LLM.
6. Otherwise the generator builds a prompt from fixed rules, the top three chunks and the question, calls the LLM, replaces citation markers with Confluence links, and stores the answer in the cache. Only answers that have sources are cached (design assumption).
7. Layer 2 checks that every cited document lies within the person's access and writes the audit record. Layer 1 posts a Block Kit message in the thread with a button for each source.

The design contains three early exits: refusal (step 2), cache hit (step 3) and fallback (step 5). Only the final branch calls the LLM, which keeps the common cases fast and inexpensive.

### 5.2 Ingestion flow

1. An administrator or a schedule starts a run for named space keys.
2. The connector pages through the Confluence REST API, retries on rate limits, and saves raw pages so that parsers can be re-run without further calls.
3. For each page, the pipeline compares an MD5 checksum with the stored value and skips unchanged pages.
4. A format handler converts the page to clean Markdown. Alpha handles wiki pages, complex tables and image links. Word, PDF, Excel and OCR handlers are Beta.
5. The text is split into chunks of 512 tokens with an overlap of 50 and tagged with space, department, file type, access level, source link and checksum.
6. The chunks are embedded and written to the vector store, replacing any earlier chunks of the same page (design assumption).

## 6. Security and Access Control

Security is enforced in layers so that a single mistake does not expose restricted text. Neither reference work specifies access control; the mechanisms below are therefore this project's own contribution. They draw on the content-filter idea in Mohammed and Aref (n.d.) only in the sense that a candidate reply is checked before it is returned.

## 6.1 Gates

| Gate | Question | Alpha | Beta |
|---|---|---|---|
| 1. Channel whitelist | Is this channel approved? | Channel IDs from whitelist.yaml; unknown channels ignored silently (design assumption) | Same |
| 2. Identity | Who is this person? | Slack email looked up in a registry table approved by an administrator | VIB single sign-on and @vib.com.vn domain check (assumed; Version B not yet specified) |
| 3. Tier | What may they see? | Tiers 1 and 2 only | Tiers 1 to 4 set the maximum access level and the department |

## 6.2 Enforcement points

- **At retrieval.** The security context becomes a filter (space, department, access level not above the tier) inside the same database query as the vector search. The retrieval gateway refuses any search without a filter (design assumption).
- **At egress.** Before display, each cited document is compared with the person's tier. A document above the tier is removed, and if nothing remains the fallback reply is used.
- **In the cache.** The cache key includes the access scope, so an answer prepared for one tier is not shown to another (design assumption).
- **In the audit log (Beta).** Each request writes one JSON line with time, email, tier, query, document class and outcome.

## 6.3 Slack roles and permissions

Slack workspace roles (for example Owner or member) manage the workspace only and do not decide what the bot reveals. The Confluence app installed in Slack provides link previews and notifications; it is not the data source. The bot reads Confluence through the REST API using its own API token, stored in an environment variable.

## 6.4 Tier mapping

A proposed mapping is project manager tier 4, developers tier 2 and the client tier 1. This is a proposal and requires the client's confirmation.

# 7. Data Ingestion and Storage

## 7.1 Storage model

Each chunk is stored as one row holding its text, its embedding vector and a JSONB metadata object. PostgreSQL with the pgvector extension indexes vectors with HNSW and filters on metadata in the same query. This single-store design follows the infrastructure choice of Boonsri and Buranasaksee (2025), who also use PostgreSQL with pgvector and a Redis cache. It is preferred here because the permission filter and the similarity ranking then run in one query.

| Metadata field | Purpose |
|---|---|
| space_key | Confluence space the page belongs to |
| department_tag | Department that may read the chunk |
| file_type | Source format (wiki, table, image, and in Beta Word, PDF, Excel) |
| access_level | Minimum tier required to read the chunk |
| source_uri | Link used for citations |
| checksum_hash | MD5 of the page content, used to detect change |

## 7.2 Processing rules

- **Chunking:** 512 tokens with an overlap of 50, so a statement cut at a boundary is still found.
- **Tables:** merged or nested cells are converted so that figures stay attached to their column headers.
- **Embedding:** the same model is used at ingestion and at query time, otherwise vectors cannot be compared.
- **Updates:** an edited page replaces its earlier chunks (design assumption). Pages deleted in Confluence leave orphan chunks, which a periodic check of page IDs could remove (proposal).
- **Staging:** raw pages are kept locally in Project A and in encrypted S3 storage in Project B.

### 7.3 Cache

The semantic cache returns a stored answer for a similar question asked within the same access scope. Entries expire after a time limit (design assumption). The team draft also describes caching search results rather than final answers; caching answers saves the LLM call as well, and one approach should be fixed in a decision record.

## 8. Evaluation Plan

The evaluation borrows its structure from Boonsri and Buranasaksee (2025) and adapts it to a retrieval system. It is a plan, not a result: no measurements have yet been taken.

### 8.1 Measures

| Measure | Method | Target |
|---|---|---|
| Answer correctness | A fixed set of test questions per language, each scored pass, partial or fail against a written checklist by two reviewers; normalised accuracy = pass + 0.5 × partial | At least 80% in English and in Vietnamese |
| Citation validity | Each cited link opens the page that supports the claim | All citations valid |
| Refusal behaviour | Questions with no answer in the corpus must return the fallback reply | No invented answers |
| Access control | Test accounts at different tiers ask the same questions; no text above the tier may appear | Zero leaks |
| Response time | Time from mention to posted answer, cache hit and miss reported separately | Under 3 seconds (Alpha target) |
| Usability | System Usability Scale questionnaire given to the client and pilot users | Mean above 68, the benchmark used in the reference study |

### 8.2 Method

Boonsri and Buranasaksee (2025) used pass, partial and fail judgements, normalised accuracy and bootstrap confidence intervals. This plan keeps those practices. Because the test set will be small, results will be reported as proportions with confidence intervals rather than significance tests alone, and partial answers will be counted as failures in the primary measure, with a sensitivity check that counts them as passes, as in the reference study.

### 8.3 Limits of the comparison

The reference study measured developer productivity with an AI coding agent on LINE and Google Sheets tasks. It did not evaluate retrieval quality or document-level permissions, so its time and cost ratios cannot be used to predict this system. Only its evaluation practices are reused.

## 9. Risks, Limitations and Open Decisions

### 9.1 Risks

| Risk | Effect | Mitigation |
|---|---|---|
| Wrong access label stamped at ingestion | Every later permission filter is wrong | Test the tagging step first, with sample pages at each level |
| Small local LLM gives weak Vietnamese answers | Misses the 80% target in Vietnamese | Test early; compare with Bedrock in Project B |
| Cache returns an answer for the wrong scope | Information leak | Include the access scope in the cache key; test across tiers |
| Free-tier limits of Slack or Confluence | Features unavailable | Confirm plan costs with the sponsor |
| Scope growth into Beta features | Alpha delivered late | Keep Beta components switchable and out of the Alpha critical path |

### 9.2 Limitations of the supporting literature

Mohammed and Aref (n.d.) is a descriptive overview with no experiments, no publication year or venue in the copy reviewed, and no treatment of LLMs or retrieval-augmented generation. Boonsri and Buranasaksee (2025) is a peer-reviewed conference paper, but its sample is small (nine developers, eight microtasks), its statistical power is stated to be

below conventional levels, and its figures are not fully consistent between the abstract and Table I. Further sources on retrieval-augmented generation and on access control in enterprise search should be added before the final report.

### 9.3 Open decisions

- Number of chunks passed to the LLM: three or five.
- Embedding model and vector dimension (for example a 1536-dimension managed model or an open-source multilingual model).
- Vector store: pgvector or another store; a decision record is due in Week 7.
- LLM choice for Project A.
- Identity design for Project B, which has not been specified.
- Whether the cache stores final answers or retrieval results.
- Confirmation of the design assumptions and the tier mapping with the client.
- Costs of any paid Slack or Confluence plan.

## 10. Conclusion

The VIB Internal Knowledge Assistant answers staff questions from Confluence through a four-layer pipeline in which security is applied before retrieval and again before display. The architecture adopts the layering, adapter and storage choices reported by Boonsri and Buranasaksee (2025), and its response stages map onto the components of the reference architecture of Mohammed and Aref (n.d.). Its distinctive elements are permission-aware retrieval, mandatory citation and a refusal path when no evidence exists. The Alpha release will be judged against the measures in Section 8, and the open decisions in Section 9 must be settled with the client before Beta.

## References

Boonsri, R & Buranasaksee, U 2025, 'A repository-first framework for democratizing chatbot development', *2025 9th International Conference on Information Technology (InCIT)*, IEEE, doi:10.1109/INCIT66780.2025.11276047.

Mohammed, M & Aref, MM n.d., *Chatbot system architecture*, Department of Computer Science, Faculty of Computer and Information Sciences, Ain Shams University, Cairo. [Year and publication venue not stated in the copy reviewed; confirm before submission.]